WhoPlay — Guia 2: Jogadores e compatibilidade

Oct 28, 2026 · @Sebastião Cardoso

Responsável sugerido: Tchali Tchalimba. Este guia cobre o coração do WhoPlay: procurar jogadores perto e calcular a compatibilidade (RF09, RF10). Inclui também a parte de matemática pedida nos objetivos do projeto: a fórmula de Haversine (método numérico) e a média e o desvio-padrão (estatística). O código está no guia principal, "WhoPlay — Guia da base de dados e da API"; aqui fica o que te cabe, por que ordem e como sabes que acabaste.

Guia principal (todo o código): WhoPlay — Guia da base de dados e da API

O que vais construir

No fim deste guia, o separador Jogadores mostra pessoas reais da base de dados, ordenadas pela compatibilidade. O Perfil do jogador explica essa compatibilidade com os 5 critérios (com o `seed.sql`, o Tchali dá 93%).

Camada	O que é teu
Ecrãs	04 Encontrar jogadores, 05 Perfil do jogador
Base de dados	Não crias tabelas: usas as do Guia 1. A tua parte são as **consultas SQL** de prática, com a fórmula de Haversine e a estatística (`GROUP BY`, `AVG`)
Rotas da API	`GET /jogadores?modalidade=1&raio=5`, `GET /jogadores/:id?modalidade=1`
Lógica	`utils/compatibilidade.js`: Haversine, 5 critérios, média e desvio-padrão

Ficheiros

Ficheiro	Criar ou alterar	Onde está o código no guia principal
`database/consultas.sql` (consultas 1, 2, 3, 4 e 8)	Criar	Parte 5

Ficheiro	Criar ou alterar	Onde está o código no guia principal
`backend/src/utils/compatibilidade.js`	Criar	Parte 9.1
`backend/src/routes/jogadores.js`	Criar	Parte 9.2
`backend/src/server.js`	Tirar o `//` da linha `/jogadores`	Parte 6.8
`lib/data/models.dart` (`Criterio`, `Jogador`)	Juntar debaixo de `// Guia 2`	Parte 12.5
`lib/services/api.dart` (método `jogadores`)	Juntar debaixo do marcador "jogadores (Guia 2)"	Parte 12.4
`lib/screens/find_players_screen.dart`	Substituir	Parte 12.12
`lib/screens/player_profile_screen.dart`	Substituir	Parte 12.13
`test/widget_test.dart`	Substituir	Parte 12.20

Quem faz o quê na equipa

Guia	Responsável sugerido	Funcionalidade
1	Sebastião	Contas e perfil + base comum
2	Tchali	Jogadores e compatibilidade
3	Hélio	Jogos e check-in
4	Cristiano	Convites, avisos e integração final

Este guia é o que melhor serve para a apresentação da parte de matemática do projeto: o exemplo do cálculo dos 93% está na Parte 9 do guia principal.

Antes de começar

- A **Fase 0 do Guia 1** tem de estar no `main`: `server.js`, `db.js`, `auth.js`, as tabelas `utilizadores`, `modalidades`, `utilizador_modalidades` e `disponibilidades` com dados, e o `api.dart` base. Sem isto não há utilizadores para comparar.
- XAMPP, Node.js e VS Code com Thunder Client, como no guia principal (Partes 1 e 11).

Cria o teu ramo depois de a Fase 0 entrar no `main`:

```
git checkout main
git pull
git checkout -b feature/jogadores
cd backend && npm install && cp .env.example .env
```

Corre o `schema.sql` e o `seed.sql` que vieram no `pull` (guia principal, Parte 4).

Passo a passo

1. SQL: perceber as contas antes de as programar

- [ ] Criar `database/consultas.sql` com o cabeçalho, o `USE whoplay` e as consultas **1, 2, 3, 4 e 8** da Parte 5. As outras usam tabelas dos Guias 3 e 4, que as acrescentam depois.
- [ ] Correr a consulta 4 no phpMyAdmin e confirmar: Tchali 1,8 km, Helio 3,1 km, Cristiano 4,2 km.
- [ ] Correr a consulta 8 e confirmar: Padel com 4 jogadores e avaliação média de 4,55.
- [ ] Fazer à mão (ou numa folha de cálculo) a conta do exemplo Sebastião × Tchali da Parte 9: 30 + 25 + 16 + 12 + 10 = 93. Vais precisar de a explicar na apresentação.

2. API

- [ ] `backend/src/utils/compatibilidade.js` (Parte 9.1).
- [ ] `backend/src/routes/jogadores.js` (Parte 9.2).
- [ ] No `server.js`, tirar o `//` da linha `app.use('/jogadores', ...)`.
- [ ] No `routes/perfil.js` (do Guia 1), voltar a pôr `const { NIVEIS } = require('../utils/compatibilidade');` no lugar da constante temporária. Combina com o Guia 1 qual dos 2 faz esta troca, para não a fazerem os 2.
- [ ] Testar no Thunder Client os pedidos 6 e 7 da Parte 11.3.

3. App

- [ ] No `models.dart`, debaixo de `// Guia 2`, juntar as classes `Criterio` e `Jogador` (Parte 12.5).
- [ ] No `api.dart`, debaixo do marcador "jogadores (Guia 2)", juntar o método `jogadores` (Parte 12.4).
- [ ] Os 2 ecrãs chamam `Api.enviarConvite`, que é do Guia 4. Se o Guia 4 ainda não tiver entregado, junta esta versão temporária debaixo do teu marcador:

```
  // TEMPORÁRIO — o Guia 4 apaga isto quando juntar a versão verdadeira.
  static Future<void> enviarConvite({required int jogadorId, required int
modalidadeId, required DateTime dataHora}) async {
    throw const ApiException('Os convites ainda não estão prontos.');
  }
```

- [ ] Substituir `find_players_screen.dart` (Parte 12.12) e `player_profile_screen.dart` (Parte 12.13).
- [ ] Substituir `test/widget_test.dart` (Parte 12.20).
- [ ] `flutter analyze` e `flutter test` sem problemas.

Não apagues o `mock_data.dart`: os ecrãs dos Guias 3 e 4 ainda o usam até às suas entregas.

Ligações com os colegas

Recebes	De quem	Enquanto não chega
Tabelas de utilizadores e disponibilidade, `api.dart` base	Guia 1 (Fase 0)	Não dá para começar: espera pela Fase 0
`Api.enviarConvite` e a rota `POST /convites`	Guia 4	Versão temporária (passo 12): o botão "Convidar" mostra "Os convites ainda não estão prontos."

Entregas	A quem
`NIVEIS` e `distanciaKm` em `utils/compatibilidade.js`	Guia 1 (`perfil.js`), Guia 3 (`jogos.js` , distância ao jogo) e Guia 4 (`convites.js` , distância de quem convida)
O ecrã Jogadores, de onde se envia o convite	Guia 4 (o `HomeShell` salta para Convites depois de "Enviar convite")

O `utils/compatibilidade.js` é usado por 3 colegas, por isso tenta entregá-lo cedo: podes fazer um primeiro Pull Request só com ele, logo depois do passo 5.

O filtro "Até 5 km" e os pedidos só funcionam para quem tem `latitude` e `longitude`. Os utilizadores do `seed.sql` já têm; os novos recebem a localização provisória no Onboarding (Guia 1).

Testes de aceitação e entrega

Com a base de dados acabada de recarregar e com o token do Sebastião:

- [ ] `GET /jogadores?modalidade=1` → Tchali 93, Helio 85, Cristiano 63; estatísticas com média 80,3 e desvio-padrão 12,7
- [ ] `GET /jogadores/2?modalidade=1` → critérios 30, 25, 16, 12 e 10
- [ ] `GET /jogadores?modalidade=1&raio=2` → só o Tchali (1,8 km)
- [ ] Um utilizador novo sem localização → 400 "Define primeiro a tua localização no perfil."
- [ ] Na app, o separador Jogadores mostra os 3 cartões: 93% e 85% em lima, 63% em cinzento
- [ ] Na app, Ténis mostra só o Helio (o único, além do Sebastião, que joga Ténis)
- [ ] Na app, "Ver perfil" do Tchali mostra as 5 barras com 30/30, 25/25, 16/20, 12/15 e 10/10
- [ ] `flutter analyze` e `flutter test` sem problemas

Entrega:

```
git add database/consultas.sql backend/src lib test
git commit -m "Jogadores e compatibilidade: consultas, API e ecrãs"
git push -u origin feature/jogadores
```

No GitHub, abre um Pull Request de `feature/jogadores` para `main`. Se o Git disser que há conflitos no `api.dart` ou no `models.dart`, confirma que escreveste só debaixo do teu marcador; se o conflito for no `server.js`, mantém as linhas dos 2 lados.